

Why Neural Networks Explode: A Deep Dive into Training Divergence

From a real experiment training protein language models from scratch

Contents

- 1. What Happened: A Cautionary Tale
- 2. Background: How Neural Network Training Works
- 3. The Learning Rate: Your Most Important Hyperparameter
- 4. Why Larger Models Are More Fragile
- 5. Batch Size and Gradient Noise
- 6. The Anatomy of Our Divergence
- 7. How to Prevent Divergence
- 8. Key Takeaways

1. What Happened: A Cautionary Tale

We were training ESM2 protein language models from scratch — six models ranging from 7.7 million to 1.47 billion parameters — all on the same dataset of 10,000 protein sequences. Our goal was simple: train every model for exactly 200 epochs (passes through the data) and compare how well each one learns.

The four smallest models (7.7M to 148M parameters) trained beautifully. Their loss curves descended smoothly and settled at a plateau around 2.59. But the 651M-parameter model told a very different story: around epoch 44, its loss suddenly shot upward from 2.68 to over 4.3 — *worse than a model that had never been trained at all*. The model had diverged.

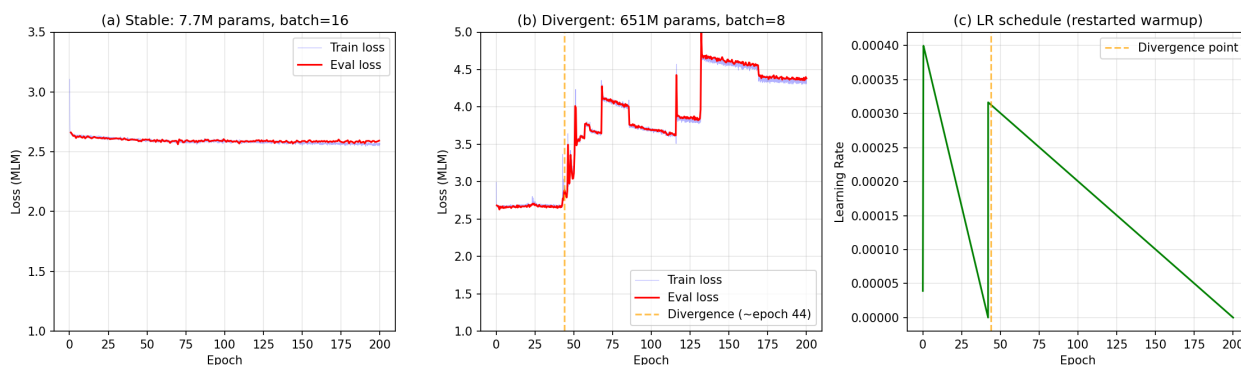


Figure 1. (a) The 7.7M model trains stably for 200 epochs, with train and eval loss converging. (b) The 651M model diverges catastrophically around epoch 44 — eval loss climbs from 2.68 to 4.38. (c) The learning rate schedule, which was accidentally restarted from zero, reaches its peak right around the divergence point.

This article explains exactly why this happened — and in doing so, teaches you the core dynamics of neural network training that every practitioner needs to understand.

2. Background: How Neural Network Training Works

Training a neural network means finding parameter values (weights) that minimize a **loss function** — a number that measures how wrong the model's predictions are. In our case, the loss function is **masked language modeling (MLM)**: we randomly hide 15% of amino acids in a protein sequence, ask the model to predict what was hidden, and measure how surprised it is by the correct answer using cross-entropy loss.

The workhorse algorithm is **stochastic gradient descent (SGD)** and its variants (we use AdamW). The key idea:

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla L(\theta_t)$$

In words: take the current parameters θ , compute the gradient of the loss (which direction makes the loss increase fastest), and step in the *opposite* direction by an amount proportional to the **learning rate** η .

Think of it like descending a mountain in fog. You can feel the slope beneath your feet (the gradient), and you take a step downhill. The learning rate controls how big each step is. Too small, and you'll take forever to reach the valley. Too big, and you might overshoot the valley entirely and end up on a higher peak on the other side.

What "Loss" Means Here

Our models output a probability distribution over the 33 possible amino acids at each masked position. The cross-entropy loss is:

$$L = - \sum_i \log P(\text{correct token}_i)$$

If the model assigns probability 1.0 to the correct amino acid, the loss is 0 (perfect). If it assigns equal probability to all 33 tokens (random guessing), the loss is $\log(33) \approx 3.50$. Our best models reach about 2.56, meaning they're significantly better than random but still far from perfect — protein sequences are genuinely hard to predict.

When we say the 651M model "diverged to 4.38," that means its predictions became *worse than random guessing*. The model didn't just forget what it learned — it learned actively wrong patterns.

3. The Learning Rate: Your Most Important Hyperparameter

The learning rate η is the single most consequential number you choose when training a neural network. To understand why divergence happens, you need to understand what happens at different learning rates.

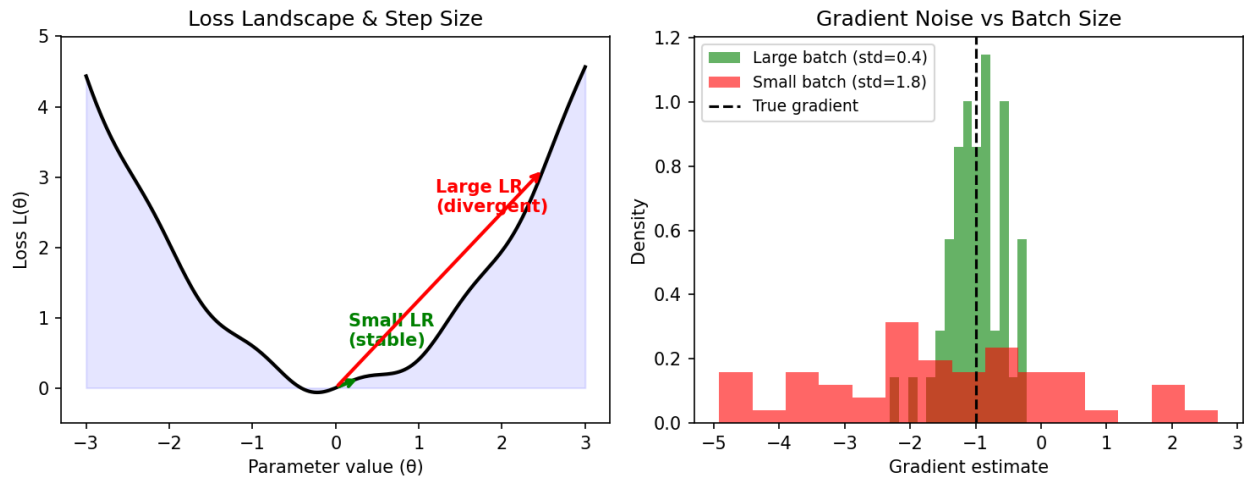


Figure 2. Left: A small learning rate keeps parameter updates within the loss basin (stable), while a large learning rate can overshoot the basin entirely (divergent). Right: Smaller batches produce noisier gradient estimates, making the effective step direction less reliable.

The Stability Threshold

For a simple quadratic loss $L(\theta) = \frac{1}{2} H \theta^2$ (where H is the curvature, also called the Hessian), the SGD update becomes:

$$\theta_{t+1} = \theta_t (1 - \eta H)$$

This converges only when $|1 - \eta H| < 1$, which means:

$$\eta < 2/H$$

This is the **stability threshold**. If the learning rate exceeds $2/H$, each step makes the parameters *larger* instead of smaller, and the loss grows exponentially. That's divergence.

The key insight: **H is not a fixed number**. The loss landscape's curvature changes as the model trains. A learning rate that was perfectly safe at step 1,000 can become catastrophically large at step 50,000 if the model has wandered into a region of high curvature.

Learning Rate Schedules

In practice, we rarely use a constant learning rate. Instead, we use a **schedule**:

Warmup: Start with a very small learning rate and gradually increase it over the first few hundred steps. This is crucial because at initialization, the gradients can be large and erratic. Warming up lets the model find a reasonable region before taking big steps.

Decay: After warmup, gradually decrease the learning rate. This lets the model make fine adjustments as it approaches a minimum, rather than bouncing around with large steps.

Our schedule uses 500 steps of linear warmup from 0 to $\eta=4\times10^{-4}$, then linear decay back to 0 over the remaining steps. This is standard and works well — *when applied correctly*.

4. Why Larger Models Are More Fragile

Our four small models (7.7M–148M parameters) all trained stably with learning rate 4×10^{-4} . The 651M model diverged with the same learning rate. Why?

More Parameters = Sharper Minima

Larger models have more parameters, which means the loss landscape is higher-dimensional. Research has shown that larger models tend to find **sharper minima** — regions where the loss surface curves steeply. In our notation, the curvature H is larger.

Remember the stability condition: $\eta < 2/H$. If H is larger for the 651M model, the same learning rate $\eta = 4\times10^{-4}$ might be safe for the 7.7M model but too large for the 651M model.

Spectral Evidence from Our Experiments

In our earlier experiments (lab entries 005 and 008), we measured the **effective rank** of attention weight matrices across model sizes. Effective rank measures how many dimensions the weights actually use — a low effective rank means the weights have collapsed into a low-dimensional subspace.

Model	Params	Effective Rank (final)
6L-320H	7.7M	0.63
12L-320H	15.1M	0.60
20L-480H	55.9M	0.58
30L-640H	148.5M	0.52
33L-1280H	651.7M	0.48

The larger models show **more rank collapse**. This means their learned representations live in a narrower subspace, which corresponds to sharper curvature in certain directions. Small perturbations along the steep directions (from a gradient step that's too large) can push the model out of this narrow valley.

5. Batch Size and Gradient Noise

There's a second critical factor in our divergence: **batch size**.

In SGD, we don't compute the gradient over the entire dataset (that would be too slow). Instead, we sample a random **mini-batch** of B examples and compute the gradient on just those examples. This gradient is an unbiased *estimate* of the true gradient, but it's noisy.

$$\text{Var}(\nabla L_{\text{batch}}) \approx \sigma^2 / B$$

The variance of the gradient estimate is inversely proportional to the batch size B . A batch of 16 produces gradient estimates that are twice as precise as a batch of 8, and four times as precise as a batch of 4.

Key insight: The 651M model used batch size 8 (due to GPU memory constraints), while the smaller models used batch size 16. This means the 651M model had **twice the gradient noise** at each step. Combined with the same learning rate, each parameter update for the 651M model was noisier and more likely to overshoot.

The Effective Learning Rate

Researchers often think about the **effective learning rate**, which accounts for batch size:

$$\eta_{\text{eff}} = \eta / B$$

For the same nominal learning rate $\eta = 4 \times 10^{-4}$:

Model	Batch Size	Effective LR (η/B)
7.7M–148M	16	2.5×10^{-5}
651M	8	5.0×10^{-5}
1,465M	4	1.0×10^{-4}

The 651M model has **double the effective learning rate** of the smaller models. The 1.47B model has **quadruple**. This is exactly the wrong direction — larger models need *smaller* effective learning rates because their loss landscape is sharper, but our setup gave them *larger* ones.

6. The Anatomy of Our Divergence

Now we can reconstruct exactly what happened to the 651M model:

Step 1: Resume with reset schedule (epoch 0). The model loaded trained weights from a previous run (already at a good loss of ~ 2.63). But a bug in our training script caused the learning rate schedule to *restart from zero* instead of continuing from where it left off.

Step 2: Warmup phase (epochs 0–0.4). The learning rate gradually increased from 0 to 4×10^{-4} . During this phase, training looked fine — the model maintained its good loss because the learning rate was still small.

Step 3: Peak learning rate (epochs 0.4–44). With η near 4×10^{-4} and $\text{batch}=8$, the effective learning rate was 5×10^{-5} . For a few epochs, the model continued learning. But the loss landscape was already sharp (the model had been pre-trained for 42 epochs), and the high learning rate was gradually pushing it toward unstable regions.

Step 4: Divergence (epoch ~ 44). The model hit a region of high curvature where $H > 2/\eta$. Instead of descending, each step pushed the parameters further from the minimum. The loss spiked from 2.68 to 2.88 in a single evaluation interval. From there, it never recovered.

Step 5: Chaotic regime (epochs 44–200). Once diverged, the model entered a chaotic regime where losses oscillated between 3.5 and 4.6 — worse than random initialization. The model's weights were essentially scrambled, containing no useful learned patterns.

Why Didn't It Recover?

The learning rate was slowly decaying during this time. You might expect the model to eventually stabilize as η decreases. But by the time η had decayed enough for stable updates, the weights were so far from any good minimum that the model would have needed to essentially re-learn everything from scratch — and the remaining lr budget wasn't enough for that.

7. How to Prevent Divergence

Our accident illustrates several best practices:

7.1 Scale the Learning Rate with Batch Size

The **linear scaling rule** (Goyal et al., 2017) says: when you change the batch size by a factor k , scale the learning rate by the same factor k . If $\text{batch}=16$ works with $\eta=4 \times 10^{-4}$, then $\text{batch}=8$ should use $\eta=2 \times 10^{-4}$, and $\text{batch}=4$ should use $\eta=1 \times 10^{-4}$.

Even better, use the **square root scaling rule** for AdamW: $\eta \propto \sqrt{B}$. This is often more stable for Adam-style optimizers.

7.2 Scale the Learning Rate with Model Size

Larger models generally need smaller learning rates. The μP (maximal update parameterization) framework (Yang et al., 2022) provides principled rules for scaling the learning rate based on model width. A rough heuristic: scale η inversely with the square root of the hidden dimension.

7.3 Monitor for Early Signs

Divergence rarely happens instantaneously. Watch for:

- **Gradient norm spikes:** Sudden increases in $\|\nabla L\|$ suggest the model is in a high-curvature region
- **Loss oscillation:** Increasing variance in per-step loss, even if the average is still decreasing
- **Eval-train gap inversion:** If eval loss starts increasing while train loss continues decreasing, the model may be entering an unstable regime

7.4 Use Gradient Clipping

Clip the gradient norm to a maximum value (e.g., 1.0). This prevents any single step from being too large, acting as a safety net even if the learning rate is slightly too high:

$$g \leftarrow g \cdot \min(1, \text{max_norm} / \|g\|)$$

7.5 Proper Checkpointing

When resuming training from a checkpoint, **always restore the full training state**: model weights, optimizer state (momentum, adaptive learning rate accumulators), learning rate schedule position, and the global step counter. Restoring only the weights — as we accidentally did — resets the schedule and optimizer, which can be catastrophic for a model that has already adapted to a specific region of the loss landscape.

8. Key Takeaways

1. Divergence is the loss increasing without bound. It happens when the learning rate exceeds the stability threshold $\eta < 2/H$, where H is the local curvature of the loss landscape.

2. Larger models are more fragile. They develop sharper loss landscapes (higher curvature), so the same learning rate that works for a small model can destroy a large one.

3. Smaller batches amplify the problem. They produce noisier gradients, and since memory-limited large models are forced to use smaller batches, this compounds with their inherent fragility.

4. The learning rate schedule matters enormously. Restarting warmup on an already-trained model can push it into unstable regions. Always resume the full training state, not just the weights.

5. Prevention is better than cure. Scale the learning rate with both batch size and model size. Use gradient clipping. Monitor gradient norms and loss variance for early warning signs.

Written from experimental observations on ESM2 protein language models trained from scratch.

Lab entries 005, 008, 010 — dl-bio project, March 2026.